The Pacer & MyLang Handbook

A Complete Guide to Scripting, Scientific Computing, and Application Development

System Administrator and Language Designer

2026

License: MIT
Language Specification Version: 2.4.0 Core

Abstract

Welcome to **MyLang**, a dynamic, high-level computational scripting language optimized for mathematical modeling, electrical engineering simulations, cryptographic prototyping, and data analysis. To provide an optimal developer workflow, MyLang is natively integrated with **Pacer**, a lightweight, graphical IDE engineered for rapid script development, execution, and debugging. This manual serves as an exhaustive, self-teaching guide for engineers, researchers, and students who want to build, test, and package applications within the Pacer and MyLang ecosystem.

Contents

License & Copyright

1 Quick Start & Environment Setup

Setting up your environment takes less than five minutes. Because Pacer and MyLang are engineered using Python and PyQt5, you can install the required dependencies using any standard package manager.

1.1 Installing Dependencies

Open your terminal or command prompt and execute the following command to download the user interface and artificial intelligence backends:

```
pip install PyQt5 anthropic
```

1.2 Setting up AI Capabilities

Pacer comes with built-in code completion, explanation, and debugging models. To activate these features, fetch an API key from Anthropic and add it to your system's environment variables:

- **macOS / Linux:** Open your terminal profile script (such as `~/.zshrc` or `~/.bashrc`) and append the following line:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

- **Windows (Command Prompt):** Set the variable directly or through system properties:

```
set ANTHROPIC_API_KEY=sk-ant-...
```

1.3 Organizing the Project Files

Ensure your project files are organized cleanly. A typical directory structure should look like this:

```
your-project/
   Pacer_mylang.py      ← The graphical editor application
   MYLANG_DOCS.md       ← Full integrated language reference
   README.md            ← Core documentation file
   mylang/              ← The core engine directory
        lexer.py        ← Tokenizer: Converts text to tokens
        parser.py       ← Parser: Builds the Abstract Syntax Tree
        interpreter.py  ← Interpreter: Executes AST nodes
        ast_nodes.py    ← Data structure representations
        stdlib.py       ← Embedded libraries (math, stats, ee, crypto)
        main.py         ← Command-line entry point & REPL handler
        examples/       ← Prebuilt scripting files
             hello.ml
             fibonacci.ml
             electrical_engineering.ml
```

1.4 Launching the Editor

With your files sorted, initialize Pacer by running:

```
python Pacer_mylang.py
```

Pacer will load immediately, greeting you with a blank tab titled `Untitled-1.ml`, fully prepared for your first block of code.

2 The Core Development Workflow

Learning a language is highly effective when you practice a rapid loop of execution and modification.

2.1 Execution Mechanics

MyLang files use the `.ml` file extension. You have multiple distinct strategies for executing scripts:

1. **Direct Execution within Pacer:** Press the `F5` hotkey or type `/run` directly into the bottom command line. This redirects standard output into Pacer's integrated color-coded pane.
2. **Terminal Interface:** Run scripts directly via the command-line engine:
   ```
   python mylang/main.py yourfile.ml
   ```
3. **Global Command Utility:** If installed locally (`pip install -e .`), call the tool globally:
   ```
   mylang yourfile.ml
   ```
4. **The Interactive REPL:** If you want to evaluate statements one line at a time, spawn an interactive session:
   ```
   mylang --repl
   ```

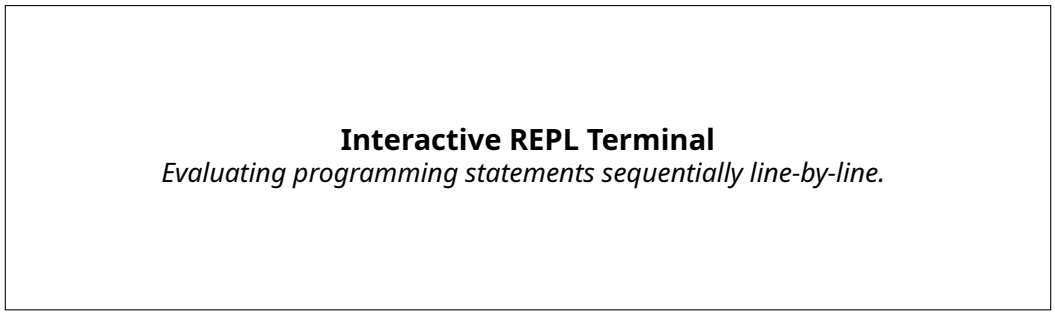

Figure 1: Visual representation of the interactive command-line environment.

2.2 Compilation and Debug Flags

If you are curious about compiler architecture or are debugging a strange syntax issue, you can inspect the intermediate evaluation states using flags:

- `mylang --tokens filename.ml` dumps the complete token stream produced by the lexer.
- `mylang --ast filename.ml` outputs a structural visual mapping of the Abstract Syntax Tree.

2.3 Integrated Editor Features

- **Default Templates:** Pressing `Ctrl + N` auto-populates a fresh, syntax-ready `.ml` template file.
- **New Other Types:** Need to build helper files? `Ctrl + Shift + N` opens options to write Python, JavaScript, HTML, or XML.
- **Unsaved Indicators:** A small marker appears inside a file tab whenever edits are made. Saving your code (`Ctrl + S`) automatically commits modifications and safety-appends `.ml` if omitted.
- **Auto-Indent:** Pressing Enter inside a block encapsulated with `{` automatically aligns indents to match nested scopes.
- **Documentation Viewer:** Clicking `mylang menu` → `Open Documentation` splits Pacer's editor pane to display the reference guide.

3 Core Language Syntax & Primitive Types

MyLang is dynamically typed. Variables assume their underlying structural behavior at runtime from the data assigned to them. Variable bindings are block-scoped, meaning nested variables inside blocks do not leak into outer environments.

3.1 Initializing Variables

Variables are declared using the `let` keyword. You only use `let` the **first time** you introduce a variable. For subsequent modifications, assign to it directly without the keyword:

```
let counter = 10;        // Declaration and assignment
counter = counter + 1;   // Re-assignment (legal without let)
```

Type	Syntax Example	Core Concept
Number	`let x = 42;`	Integers and floating-points are unified seamlessly. Supports scientific notation (e.g. `1e-6`, `4.7e-12`).
String	`let name = "Alice";`	Textual data encapsulated within double quotation marks exclusively.
Boolean	`let active = true;`	Flag states, adopting either `true` or `false` representations.
Null	`let result = null;`	Denotes the absolute intentional absence of any value.
Array	`let items = [1, 2, "3"];`	An ordered, zero-indexed sequence capable of mixing data types.
Hash	`let data = {"id": 1};`	Key-value associative dictionaries for looking up fields by name.
Complex	`let z = complex(3, 4);`	Represents mathematical numbers with imaginary parts ($3 + 4j$).
Matrix	`let m = matrix([[1,2],[3,4]]);`	Two-dimensional structural numeric grid for linear algebra.
Function	`let f = fn(x) {...};`	Functions are first-class values and can be passed as data.

Table 1: The core data types native to the MyLang execution environment.

3.2 Data Type Catalog

Check types at runtime using the `type()` function:

```
print(type(42));          // number
print(type("hello"));     // string
```

3.3 String Built-in Methods

Strings in MyLang include built-in methods for cleaning, searching, splitting, replacing, and extracting text. This is exceptionally useful for loading external CSV tables, logs, or user interfaces:

- `str.upper()`: Casts a string to UPPERCASE.
- `str.lower()`: Casts a string to lowercase.
- `str.trim()`: Trims leading and trailing whitespace.
- `str.split(sep)`: Splits string on separator into an array.
- `str.replace(old, new)`: Replaces substrings in place.
- `str.contains(sub)`: Returns `true` if substring exists.
- `str.starts_with(prefix)`: Evaluates prefix matches.
- `str.ends_with(suffix)`: Evaluates suffix matches.
- `str.slice(start, end)`: Extracts substring across a specified index range.
- `str.index_of(sub)`: Returns index of matching character or -1.

```
let raw = "   Platform-Engine-2026 ";
let clean = raw.trim();
let segments = clean.split("-");
print(segments[0]); // Output: Platform
```

4 Control Flow Structures

Control flow tells your software when to make decisions and when to cycle through repetitive operations.

4.1 Conditional Evaluation

Conditions are evaluated from top to bottom. The first block that evaluates to `true` runs, while subsequent links are bypassed.

```
let measurement = 85;

if (measurement >= 90) {
    print("Excellent Status");
} else if (measurement >= 80) {
    print("Normal Operational Status");
} else {
    print("Warning: Threshold Dropped");
}
```

4.2 Definite Iteration (for-in)

The `for-in` loop is optimized to cycle over collections, strings, hashes, or ranges without needing an independent index variable.

```
// Iterating over standard arrays
for (color in ["red", "green", "blue"]) {
    print(color);
}

// Running across numerical ranges with range(start, stop, step)
for (i in range(0, 15, 5)) {
    print(i); // Outputs 0, 5, 10
}

// Accessing associative keys inside a hash
let setup = {"host": "127.0.0.1", "port": 9000};
for (key in setup) {
    print(key + " maps to " + str(setup[key]));
}
```

4.3 Indefinite Iteration (while)

Use `while` loops when you are unsure exactly how many times an operation must run, continuing until a boolean exit condition triggers.

```
let index = 1;
while (index <= 3) {
```

```
    print("Loop step: " + str(index));
    index = index + 1; // Critical to modify state to avoid infinite loops!
}
```

5 Designing Functions & Closures

Functions package operations into modular, reusable blocks of code.

5.1 Syntax and Parameters

Declare a function with the `fn` keyword, parameters inside parentheses, and code enclosed in curly braces.

```
fn compute_power(current, resistance) {
    return (current * current) * resistance;
}

let system_power = compute_power(1.5, 220);
print("Total Watts Dissipated: " + str(system_power));
```

5.2 Closures and First-Class Functions

Because functions are first-class objects, you can return a nested function from a parent function. This inner function retains access to variables declared in its parent environment—a pattern known as a **closure**.

```
fn create_amplifier(multiplier) {
    return fn(signal) {
        return signal * multiplier;
    };
}

let gain_of_three = create_amplifier(3);
print(gain_of_three(15)); // Outputs: 45
```

5.3 Functional Arrays: Map, Filter, and Reduce

MyLang natively supports modern chainable arrays using higher-order functional operations:

```
let baseline = [1, 2, 3, 4, 6, 9];

// Transform every element
let squares = baseline.map(fn(x) { return x * x; }); // [1, 4, 9, 16, 36, 81]

// Keep elements matching a boolean condition
let evens = baseline.filter(fn(x) { return x % 2 == 0; }); // [2, 4, 6]

// Accumulate into a single output value
let total_sum = baseline.reduce(fn(acc, x) { return acc + x; }, 0); // 25
```

6 Built-In Data Structures: Arrays & Hashes

Data structures organize data in memory so your scripts can access and manipulate information efficiently.

6.1 Arrays

Arrays are ordered, dynamic, zero-indexed lists.

6.1.1 Global Array Operations

- `len(arr)`: Evaluates array element count.
- `push(arr, val)`: Appends element to tail.
- `pop(arr)`: Pops and returns the tail element.

6.1.2 Native Array Methods

- `arr.len()`: Evaluates length.
- `arr.push(val)`: Appends inline.
- `arr.pop()`: Removes and returns last element.
- `arr.contains(val)`: Checks value existence, returning a boolean.
- `arr.index_of(val)`: Locates index position or returns -1.
- `arr.join(sep)`: Collapses array into string using delimiter.
- `arr.reverse()`: Clones a reversed version of the array.
- `arr.slice(start, end)`: Partitions a segment of the array.

6.2 Hash Maps

Hashes are key-value dictionary tables. Keys must be primitives (strings, numbers, booleans).

6.2.1 Native Hash Methods

- `hash.len()`: Counts key-value pairs.
- `hash.keys()`: Returns array of keys.
- `hash.values()`: Returns array of values.
- `hash.has(key)`: Checks key existence.
- `hash.del(key)`: Removes key-value entry from workspace.

```
let circuitMap = {
    "transistor": "NPN",
    "inductors": 5
};
if (circuitMap.has("transistor")) {
    print("Transistor Type: " + circuitMap["transistor"]);
}
```

7 Special Scientific Data Types

MyLang features specialized internal objects designed to run computational math directly on hardware virtual scopes without requiring heavy third-party libraries.

7.1 Complex Numbers

Ideal for AC circuit analysis, phase angles, and waveform dynamics.

7.1.1 Instantiation & Properties

Declare complex numbers using the literal `j` suffix or the `complex(real, imag)` function:

```
let z1 = 3 + 4j;
let z2 = complex(1, -2);
```

7.1.2 Native Complex Methods

- `z.real()`: Real component value.
- `z.imag()`: Imaginary component coefficient.
- `z.abs()`: Magnitude calculation, equivalent to $\sqrt{\text{real}^2 + \text{imag}^2}$.
- `z.conj()`: Conjugate sign invert ($a + bj \leftrightarrow a - bj$).
- `z.angle()`: Computes phase angle directly in degrees.

```
let wavePhase = complex(10, -10);
print(wavePhase.abs());   // Magnitude: 14.142136
print(wavePhase.angle()); // Angle in Deg: -45.00
```

7.2 Math Matrices

MyLang integrates a Row-major multi-dimensional matrix layout compiler capable of solving complex linear systems.

7.2.1 Matrix Constructors

- `matrix(nested_arr)`: Declares explicit matrix coordinates.
- `mat_zeros(rows, cols)`: Creates matrix prefilled with zeros.
- `mat_identity(n)`: Formats square $n \times n$ identity matrix.

7.2.2 Native Matrix Methods & Functions

- `M.shape()` or `mat_shape(M)`: Returns hash grid dimensions `{"rows": r, "cols": c}`.
- `M.get(r, c)` or `mat_get(M, r, c)`: Retrieves value at indexed coordinate.
- `M.set(r, c, val)` or `mat_set(M, r, c, val)`: Assigns value to coordinate.
- `M.transpose()` or `mat_transpose(M)`: Rotates row data into column data.
- `M.det()` or `mat_det(M)`: Calculates determinant.

- `M.trace()` or `mat_trace(M)`: Calculates sum of diagonal values.
- `M.scale(factor)` or `mat_scale(M, factor)`: Scalar multiplication.

```
let M = matrix([[2, 1], [1, 3]]);
print("Determinant: " + str(M.det())); // Output: 5
```

8 Global Namespace Standard Libraries

Standard libraries provide grouped namespaces of pre-written logic to manage specific calculations.

8.1 The math Library

The `math.*` library provides algebraic, trigonometric, and arithmetic routines.

8.1.1 Constants

- `math.PI` (or `PI`): $\pi \approx 3.1415926535$
- `math.E` (or `E`): $e \approx 2.7182818284$
- `math.TAU` (or `TAU`): $2\pi \approx 6.2831853071$
- `math.PHI` (or `PHI`): Golden ratio ≈ 1.6180339887
- `math.INF`, `math.NAN`: Boundaries for Infinity and undefined calculations.

8.1.2 Core Operations

- `math.sqrt(x)`: Computes square root (returns complex numbers for negative values).
- `math.pow(base, exp)`: Power function.
- `math.abs(x)`: Absolute value.
- `math.floor(x)`, `math.ceil(x)`: Rounding limits.
- `math.round(x, places)`: Decimal precision.
- `math.clamp(val, min, max)`: Constrains value to bounds.
- `math.sin(x)`, `math.cos(x)`, `math.tan(x)`: Trigonometric functions (radians).
- `math.deg(rad)`, `math.rad(deg)`: Angle conversions.
- `math.exp(x)`: Exponential power.
- `math.log(x, base)`: Computes logarithmic value. Default is natural log base.
- `math.factorial(n)`: Permutation calculations.
- `math.comb(n, k)`, `math.perm(n, k)`: Combinatorial selection.
- `math.gcd(a, b)`, `math.lcm(a, b)`: Division parameters.
- `math.random()`: Delivers LCG pseudo-random values between 0.0 and 1.0.
- `math.rand_int(min, max)`: Returns randomized integer within range.

- `math.quadratic(a, b, c)`: Evaluates roots for $ax^2 + bx + c = 0$. Outputs an array of solved real or complex numbers.

8.2 The stats Library

Ideal for data point modeling, normalization, and datasets.

- `stats.mean(arr)`: Computes arithmetic mean.
- `stats.median(arr)`: Finds middle value.
- `stats.variance(arr)`: Computes sample variance.
- `stats.stdev(arr)`: Evaluates sample standard deviation.
- `stats.normalize(arr)`: Min-Max normalizes data arrays.
- `stats.linreg(xs, ys)`: Solves the linear equation $y = mx + b$. Plots a regression model directly in the MyLang Graphing View.
- `stats.histogram(arr, bins)`: Generates frequency counts categorized into bin groups.

8.3 The ee Library

Engineered for analog circuits, impedance, filters, and AC waveform analytics.

- `ee.voltage(i, r)`, `ee.current(v, r)`, `ee.resistance(v, i)`: Ohm's Law components.
- `ee.series(arr)`, `ee.parallel(arr)`: Standard loop series/parallel resistance.
- `ee.cap_series(arr)`, `ee.cap_parallel(arr)`: Capacitance equations.
- `ee.ind_series(arr)`, `ee.ind_parallel(arr)`: Inductance equations.
- `ee.xc(f, c)`, `ee.xl(f, l)`: AC Reactance values.
- `ee.impedance_rlc(r, f, l, c)`: Models and plots RLC impedential components.
- `ee.resonant_freq(l, c)`: Resonant circuit frequency where $f_0 = 1/(2\pi\sqrt{LC})$.
- `ee.rc_charge(v0, t, r, c)`: Time-domain charging voltage where $V(t) = V_s(1 - e^{-t/RC})$.
- `ee.phasor(mag, deg)`: Converts phasor amplitude and angle to complex numbers.

8.3.1 Physical Constants

- `ee.EPSILON0`: $\epsilon_0 \approx 8.854 \times 10^{-12}$ F/m (Permittivity of free space)
- `ee.MU0`: $\mu_0 \approx 1.257 \times 10^{-6}$ H/m (Permeability of free space)
- `ee.ELECTRON`: $e \approx 1.602 \times 10^{-19}$ C (Elementary charge)
- `ee.BOLTZMANN`: $k_B \approx 1.381 \times 10^{-23}$ J/K
- `ee.PLANCK`: $h \approx 6.626 \times 10^{-34}$ J·s
- `ee.C_LIGHT`: $c \approx 299,792,458$ m/s

8.4 The crypto Library

The security module contains operations for encryption pipelines and authentication integrity verification.

- `crypto.sha256(txt)`: Computes SHA-256 hexadecimal hash.
- `crypto.hmac(msg, key)`: Computes HMAC-SHA256 signature hashes.
- `crypto.pbkdf2(pwd, salt, iter)`: Derives key token via PBKDF2 parameters.
- `crypto.encrypt_aes(txt, key)`: Encrypts plain text string to hexstream.
- `crypto.decrypt_aes(hex, key)`: Decrypts symmetric hexstream.

8.4.1 Secure Memory Vault

To prevent sensitive credentials (e.g., API tokens, passphrases) from sitting in standard global variable scopes, MyLang routes variables into an encrypted micro-memory layer:

- `crypto.secure_store(label, secret)`: Encrypts and locks secret inside local system secure buffers.
- `crypto.secure_retrieve(label)`: Temporary decryption stream to execute authentication processes.
- `crypto.secure_wipe(label)`: Garbage-collection step to sanitize workspace memory caches.

8.5 The image Library

Enables the generation of customized vector layouts, coordinate sweeps, and custom graphic boards rendering in Pacer's visual terminal tab.

- `image.blank(w, h, hex_bg)`: Sets up canvas viewport.
- `image.rect(x, y, w, h, hex_fill)`: Draws solid rectangles.
- `image.circle(cx, cy, r, hex_fill)`: Draws circles.
- `image.line(x1, y1, x2, y2, hex_stroke, width)`: Draws line segments.
- `image.text(msg, x, y, size, hex_color)`: Annotates text strings.
- `image.show(title)`: Displays canvas board in visual GUI pane.
- `image.load(url, title)`: Pulls remote vector images into target canvases.

8.6 The csv Library

Routines to serialize, format, and load structural spreadsheet layouts.

- `csv.parse(raw_string)`: Compiles CSV file sheets into multi-nested arrays.
- `csv.stringify(nested_arr)`: Exports data structures back to flat CSV strings.

9 Compiler Target Switches

MyLang integrates a multi-platform distribution and compilation layer. Based on the target compiler flag assigned at launch, you can export your completed script directly to binary, mobile, web, or standalone desktop interfaces:

- **Android Standalone Application (`apk`):** Builds virtual bytecode bundled directly into an Android wrapper package.
- **Desktop Standalone Client (`nsis` / `msi`):** Generates full Windows setup installation bundles. To avoid Windows SmartScreen warnings, you can specify your signing credentials inside PowerShell prior to invoking:

```
$env:CSC_LINK="C:\Certs\developer-key.pfx"
$env:CSC_KEY_PASSWORD="SecurePassphraseHex"
```

- **Web Canvas Engine (`html`):** Compiles your logic into a highly optimized index HTML file running inside HTML5 Canvas contexts.
- **Python Native Script (`pyw`):** Compiles layout frames into stand-alone Python Tkinter applications executing on standard offline desktop clients.

10 Dynamic Tutorials & Real-World Projects

The following tutorials combine syntax and library features into comprehensive, ready-to-run programs. copy and paste these examples directly into your Pacer editor to experiment with them.

10.1 Tutorial 1: IoT Solar Power Budget Governor

Runs calculated checks on household energy storage, prioritizing solar panel inputs and managing battery levels to avoid drainage.

```
print("=== SMART HOME RESILIENT ENERGY OPTIMIZER ===");
let solarInput = 1250.5;         // Real-time solar panel inflow (Watts)
let batteryRemaining = 1450.0;   // Stored reserves capacity (Watt-hours)

// Appliances load list structures
let appliances = [
  {"name": "Heat Pump", "load": 950.0, "priority": 1, "status": "active"},
  {"name": "Kitchen Fridge", "load": 180.0, "priority": 1, "status": "active"},
  {"name": "Living Room TV", "load": 300.0, "priority": 2, "status": "active"},
  {"name": "Outdoor Sprinkler", "load": 400.0, "priority": 3, "status": "active"
     }
];

// Calculate total active load
let totalActiveWatts = 0.0;
for (item in appliances) {
  if (item["status"] == "active") {
    totalActiveWatts = totalActiveWatts + item["load"];
  }
}
print("Total Household Power Demand: " + totalActiveWatts + " W");
print("Net Grid Intake Balance: " + (solarInput - totalActiveWatts) + " W");

// Prevent battery run-down during low solar periods
if (batteryRemaining < 1500.0) {
  print("    LOW STORAGE RESERVES: ENFORCING PRE-EMPTIVE SHUTDOWN ON LOW PRIORITY
      UNITS!");

  for (item in appliances) {
    if (item["priority"] >= 2 && item["status"] == "active") {
      item["status"] = "disabled";
      print("  [DISCONNECT] Shifting off appliance load: " + item["name"]);
    }
  }
} else {
  print("    Standby battery capacity is within nominal parameters.");
}
```

10.2 Tutorial 2: Secure Master Password Derivation

Applies modern cryptographic rules (using KDF derivation and HMAC hashing) to safely secure application access, then securely wipes memory layers to prevent credential leaks.

```
print("=== CENTRAL SECURE VAULT ACCESS GATEWAY ===");
let rawMasterValue = "AdminMasterPassphrase2026!";
```

```
let staticEntropySalt = "sha256_vector_salt_entropy_9329";

// 1. Derive strong PBKDF2 vault key (execute 1,000 hashing rounds)
let vaultKeyToken = crypto.pbkdf2(rawMasterValue, staticEntropySalt, 1000);
print("Derivation Token solved successfully.");

// 2. Lock derived token securely inside volatile memory bounds
crypto.secure_store("active_session_token", vaultKeyToken);
print("Session Token cached inside protected vault.");

// 3. Retrieve and complete validation tests
let retrievedToken = crypto.secure_retrieve("active_session_token");
let hmacVerification = crypto.hmac("LAUNCH_TELEMETRY_PIPELINE", retrievedToken);
print("System Verification Hash: " + hmacVerification);

// 4. Critical: Explicitly wipe session variables from memory
crypto.secure_wipe("active_session_token");
print("Verification complete. Memory sanitized successfully.");
```

10.3 Tutorial 3: Dynamic Circuit Schematic Illustrator

Renders an annotated electrical blueprint directly inside the MyLang Graphics Panel.

```
// Renders an annotated electrical schematic diagram
image.blank(500, 300, "#080c10");

// Draw outline card
image.rect(10, 10, 480, 280, "#0d1117");

// Draw component connections (R - L - C in series)
image.line(30, 150, 100, 150, "#58a6ff", 2); // Left input lead

// Resistor symbol (zig-zag points mapped as lines)
image.line(100, 150, 115, 130, "#3fb950", 2);
image.line(115, 130, 135, 170, "#3fb950", 2);
image.line(135, 170, 155, 130, "#3fb950", 2);
image.line(155, 130, 170, 150, "#3fb950", 2);
image.line(170, 150, 240, 150, "#58a6ff", 1); // Mid connection lead

// Inductor coils
image.circle(260, 150, 15, "#a371f7");
image.circle(285, 150, 15, "#a371f7");
image.line(300, 150, 360, 150, "#58a6ff", 1); // Second connection lead

// Capacitor plate 1 and 2
image.line(360, 120, 360, 180, "#fff", 3);
image.line(380, 120, 380, 180, "#fff", 3);
image.line(380, 150, 470, 150, "#58a6ff", 2); // Ground output lead

// Text annotations
image.text("MyLang Circuit Schematic Sim", 25, 25, 14, "#ff7b72");
image.text("R = 100 Ohm", 95, 80, 11, "#3fb950");
image.text("L = 220 mH", 240, 80, 11, "#a371f7");
image.text("C = 10 uF", 350, 80, 11, "#fff");

image.show("Series RLC Blueprint");
print("Schematic vector loaded successfully into the Graphics tab!");
```

10.4 Tutorial 4: File Parsing with CSV Text

This tutorial shows how to treat raw file-style text as structured records. The example parses CSV data, separates the header row, converts numeric fields, and prints a clean summary.

```
print("=== CSV FILE PARSING TUTORIAL ===");
let rawCsv = "device,zone,watts\nFridge,Kitchen,180\nPump,Basement,950\nRouter,
    Office,35";
let rows = csv.parse(rawCsv);
let header = rows[0];

print("Columns discovered: " + header.join(", "));
print("Data rows found: " + (len(rows) - 1));

let totalWatts = 0;
for (index in [1, 2, 3]) {
  let row = rows[index];
  let deviceName = row[0];
  let zoneName = row[1];
  let watts = row[2];
  totalWatts = totalWatts + watts;
  print(deviceName + " in " + zoneName + " uses " + watts + " W");
}
print("Total parsed load: " + totalWatts + " W");
```

10.5 Tutorial 5: Data Analysis with Statistics

This tutorial introduces a practical analysis workflow: store measurements in arrays, calculate descriptive statistics, normalize the data, and flag unusual readings.

```
print("=== SENSOR DATA ANALYSIS TUTORIAL ===");
let timestamps = [1, 2, 3, 4, 5, 6, 7];
let readings = [22.1, 23.4, 22.9, 24.2, 31.8, 23.1, 22.7];

let average = stats.mean(readings);
let medianValue = stats.median(readings);
let spread = stats.stdev(readings);
let normalized = stats.normalize(readings);

print("Average reading: " + average);
print("Median reading: " + medianValue);
print("Standard deviation: " + spread);
print("Normalized readings: " + normalized);

let alertLimit = average + (spread * 2);
for (i in [0, 1, 2, 3, 4, 5, 6]) {
  if (readings[i] > alertLimit) {
    print("Alert: reading " + readings[i] + " at time " + timestamps[i] + " is
        unusually high.");
  }
}

let trend = stats.linreg(timestamps, readings);
print("Trend model: " + trend);
```

10.6 Tutorial 6: RC Low-Pass Filter Analysis

Calculates RC cutoff points and logs frequency response parameters.

```
let R  = 1000;      // 1 kOhm
let C  = 1e-6;      // 1 uF
let f3 = 1 / (2 * PI * R * C);
print("Cutoff frequency: " + str(round(f3, 2)) + " Hz");
print("Time constant:    " + str(ee.rc_tau(R, C) * 1000) + " ms");
print("");
print("Frequency response:");
let freqs = [10, 50, 100, 159, 500, 1000, 5000];
for (f in freqs) {
    let Xc   = ee.xc(f, C);
    let Z    = sqrt(R * R + Xc * Xc);
    let gain = Xc / Z;
    let db   = ee.to_db(gain);
    print(str(f) + " Hz -> " + str(round(db, 2)) + " dB");
}
```

10.7 Tutorial 7: RLC Resonance Sweep

Analyzes frequency limits, Q factor, and resonance bandwidth of RLC circuits.

```
let R  = 50;
let L  = 10e-3;
let C  = 1e-6;
let f0 = ee.resonant_freq(L, C);
let Q  = f0 * L / R;          // Q = wOL/R for series circuit
let BW = ee.bandwidth(f0, Q);
print("Resonant frequency: " + str(round(f0, 2)) + " Hz");
print("Q factor:           " + str(round(Q, 2)));
print("Bandwidth:          " + str(round(BW, 2)) + " Hz");
print("Lower -3dB:         " + str(round(f0 - BW/2, 2)) + " Hz");
print("Upper -3dB:         " + str(round(f0 + BW/2, 2)) + " Hz");
```

10.8 Tutorial 8: 2-D Coordinate Rotation using Matrices

Rotates a coordinate vector point across rotational planes using a generated 2D rotation matrix.

```
// Rotate a point (x, y) by angle theta using a rotation matrix
fn rotation_matrix(theta_deg) {
    let theta = rad(theta_deg);
    return matrix([
        [cos(theta), -sin(theta)],
        [sin(theta),  cos(theta)]
    ]);
}
fn rotate_point(x, y, angle_deg) {
    let Rm = rotation_matrix(angle_deg);
    let pt = matrix([[x], [y]]);    // column vector
    let rp = mat_mul(Rm, pt);
    return {
        "x": round(mat_get(rp, 0, 0), 6),
        "y": round(mat_get(rp, 1, 0), 6)
    };
}
```

```
let p = rotate_point(1, 0, 90);
print("Rotated: (" + str(p["x"]) + ", " + str(p["y"]) + ")"); // Output: (0, 1)
```

10.9 Tutorial 9: Fibonacci Generator using Closures

Demonstrates persistent state containment within nested variables using a functional generator closure pattern.

```
// Generator-style Fibonacci using a closure
fn make_fib() {
    let a = 0;
    let b = 1;
    return fn() {
        let val = a;
        let tmp = a + b;
        a = b;
        b = tmp;
        return val;
    };
}
let next = make_fib();
let sequence = [];
let i = 0;
while (i < 12) {
    push(sequence, next());
    i = i + 1;
}
print(sequence);
// Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

11 Working with AI Automation inside Pacer

Pacer embeds intelligent assistive commands directly into its command console to accelerate your workflow. These features are custom-tailored to MyLang's core grammar rules, preventing the AI from recommending non-functional patterns from other environments.

11.1 Command Interface Cheat Sheet

12 Troubleshooting and Self-Debugging

When your program encounters a structural breakdown, read the output window carefully from left to right to isolate the mistake.

12.1 The Error Matrix

12.2 Formatting Reference Code Check

Avoid Python-style structural patterns (like colons or unbraced blocks). Keep your structures clean using these absolute baseline rules:

```
// CORRECT PATTERNS
let score_total = 100;           // Semicolons are mandatory at statement ends!
fn square(n) { return n * n; }   // Braces must encapsulate function blocks.
```

Console Syntax	Feature Action	Operational Behavior
`/run`	Code Execution	Compiles and evaluates the current text buffer into the console window.
`/debug`	Deep Diagnosis	Scans active script blocks and lists syntax or logical bugs.
`/fix`	Direct Repair	Generates clean replacement blocks to fix compiler breaks.
`/complete`	Code Completion	Autocomplete missing structural segments of your logic.
`/explain`	Line-by-Line Tracing	Breaks down deep loop logic or complex engineering equations into plaintext.
`/mylang <q>`	Dynamic Reference	Query the standard library documentation directly using natural language.

Table 2: Command console features integrated into Pacer.

Error Identity	Core Cause	Resolution Step
`ParseError`	Missing value or token before statement end.	Check syntax. Did you mi nal semicolon (`;`) or match brace (`}`)?
`Undefined variable`	Name referenced before instantiation.	Ensure you declare variable `let` keyword before reading
`Index out of bounds`	Index falls outside array length.	Guard lookups by checking t stay strictly less than `len(ar`
`Division by zero`	Evaluation of a zero denominator.	Add validation: `if (deno 0) {...}`.
`'foo' is not callable`	Reference does not point to a declared function.	Check syntax naming and ve `foo` is parsed beforehand.
`LexerError`	Script contains illegal characters.	MyLang uses clean ASCII va sure you avoid using extern like `@`, `#`, `$`.

Table 3: Troubleshooting and error diagnostics reference.

```
if (x > 0) { print("Positive"); }// If-logic requires conditional grouping.

// INCORRECT BLOCKS
let score_total = 100             // WRONG: Missing structural trailing semicolon
    .
if (x > 0) print("Positive")      // WRONG: Missing wrapping curly braces.
def add(a, b): return a + b       // WRONG: Python-style def/colons are illegal.
```